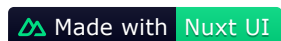


HomeCine Front

Interface web de contrôle pour un serveur audio maison multi-sources. Permet de gérer les lecteurs audio (Spotify, Deezer, radio, fichiers locaux, entrée audio), les égaliseurs paramétriques, les appareils de sortie et la diffusion réseau (VBAN / Snapcast).



Stack technique

Catégorie	Technologie
Framework	Nuxt 4 + Vue 3
UI	Nuxt UI 4 (Tailwind CSS v4)
Langage	TypeScript
HTTP	Axios (src/lib/https.ts)
Temps réel	WebSocket natif (multi-canaux)
Utilitaires	VueUse (useResizeObserver , useDebounceFn , useEventListener)
Icons	Lucide, Material Symbols, MDI, Simple Icons
Package manager	pnpm

Prérequis

- Node.js >= 20
 - pnpm >= 10
 - Serveur backend **HomeCine-Serv** accessible (API REST + WebSocket)
-

Installation

```
pnpm install
```

Configuration

Les URLs du backend sont définies dans [app/src/config.ts](#) :

```
API_URL: 'http://192.168.1.40:3007' // API REST
WS_URL:  'ws://192.168.1.40:8099'  // WebSocket
```

Variable d'environnement alternative pour le WebSocket :

```
WS_BASE=ws://192.168.1.40:8099
```

Démarrage

```
# Développement (localhost)
pnpm dev

# Développement (exposé sur le réseau local)
pnpm host

# Build production
pnpm build

# Prévisualisation du build
pnpm preview
```

Structure du projet

```
app/
├─ pages/
│   ├── login.vue           # Authentification
│   ├── index.vue           # Tableau de bord (Maison)
│   ├── spotify.vue         # Interface Spotify complète
│   └─ eq.vue               # Égaliseur paramétrique (bandes + ON/OFF +
suppression)
│   ├── eqconfig.vue        # Presets EQ (CRUD, mapping canaux)
│   ├── eq/preset.vue       # Gestion des presets EQ
│   └─ lecteurs/
│       ├── index.vue       # Liste des lecteurs audio
│       ├── new.vue         # Création d'un lecteur
│       └─ [id].vue         # Édition d'un lecteur
│   ├── devices/
│       ├── index.vue       # Liste des appareils de sortie
│       └─ [id].vue         # Détail / contrôle d'un appareil
│   ├── snap.vue            # Diffusion Snapcast
│   ├── bt.vue              # Contrôle Bluetooth
│   ├── control/index.vue   # Panneau de contrôle général
│   └─ settings/
│       ├── index.vue       # Paramètres généraux
│       ├── members.vue     # Gestion des membres
│       └─ spotify.vue      # Auth & config Spotify
```

```

├── snap.vue                # Config Snapcast
├── notifications.vue
└── security.vue

├── components/
│   ├── spotify/
│   │   ├── components/
│   │   │   ├── HomeView.vue        # Accueil Spotify (shortcuts, sections
│   │   │   │   scrollables, recherche)
│   │   │   ├── LibrarySidebar.vue  # Sidebar bibliothèque (playlists, albums,
│   │   │   │   artistes)
│   │   │   ├── LibraryList.vue     # Liste générique bibliothèque
│   │   │   ├── PlaylistList.vue    # Liste de playlists
│   │   │   ├── PlaylistRow.vue     # Ligne de playlist
│   │   │   ├── ItemPlaylist.vue    # Vue détail playlist / titres likés
│   │   │   ├── ItemAlbum.vue       # Vue détail album
│   │   │   └── ItemArtist.vue      # Vue détail artiste (top titres +
│   │   │   │   discographie)
│   │   │   ├── ItemLiked.vue       # Vue titres likés
│   │   │   └── lecture.vue         # Lecteur fixe en bas (desktop + mobile)
│   │   └── spotify.vue             # Composant Spotify legacy
│   └── lecteur/
│       ├── services/                # Formulaires par type de lecteur
│       │   ├── SpotifyServiceConfig.vue
│       │   ├── DeezerServiceConfig.vue
│       │   ├── RadioServiceConfig.vue
│       │   ├── LocalServiceConfig.vue
│       │   └── LocalInputServiceConfig.vue
│       └── transports/              # Sous-configs transport audio
│           ├── LocalStreamConfig.vue
│           ├── StreamOutFifoConfig.vue
│           └── VbanConfig.vue
│   ├── DeviceSlideover.vue         # Panneau contrôle appareil
│   ├── DeviceSpotifySlideover.vue  # Panneau appareils Spotify
│   ├── DeviceAddSlideover.vue      # Ajout appareil
│   └── ...

├── composables/
│   ├── useAuth.ts                  # Authentification (login, token, logout)
│   ├── useLecteursWs.ts            # États lecteurs temps réel (WebSocket)
│   ├── useDeviceControlWs.ts       # Contrôle appareils (WebSocket)
│   ├── useSpotifyPlayerWs.ts       # État lecteur Spotify temps réel
│   └── (WebSocket)
│       ├── useSnapWs.ts             # État Snapcast (WebSocket)
│       ├── useDeviceBus.ts          # Bus audio des appareils
│       ├── useParametricEq.ts       # Gestion des bandes EQ
│       ├── useLikedSync.ts          # Sync titres likés Spotify
│       ├── useToastHelpers.ts       # Helpers notifications toast
│       ├── useWs.ts                 # WebSocket générique
│       └── useDashboard.ts          # État global UI (panneaux, sliders)

├── src/
│   └── api/

```

```

├── ┌───┴───┐
│   │   │   │
│   │   ├── lecteur.ts           # CRUD lecteurs
│   │   └── eq.ts                # CRUD presets EQ
│   └── ┌───┴───┐
│       │   │   │
│       │   ├── lib/https.ts     # Instance Axios + intercepteur auth
│       │   └── config.ts        # URLs API et WebSocket
│
├── ┌───┴───┐
│   │   │   │
│   │   ├── utils/
│   │   │   ├── audioLayouts.ts  # 60+ layouts audio (mono → 22.2,
│   │   │   │   │               ambisonics)
│   │   │   ├── lecteurOptions.ts # Options par type de lecteur
│   │   │   └── eqTools.ts        # Calcul de courbes EQ
│   └── ┌───┴───┐
│       │   │   │
│       │   ├── assets/css/
│       │   │   ├── main.css      # CSS global (thème, overflow, variables EQ)
│       │   │   └── sliders-many-colors.css # Styles curseurs multi-couleurs
│       └── ┌───┴───┐
│           │   │   │
│           │   ├── layouts/
│           │   │   └── default.vue # Layout global (sidebar + search + main)
│           └── ┌───┴───┐
│               │   │   │
│               ├── types/
│               │   └── lecteur.ts # Types TypeScript (Lecteur, ConfEq, etc.)

```

Pages principales

/spotify

Interface Spotify complète :

- **Sidebar bibliothèque** (desktop fixe, mobile en drawer)
- **Accueil** : shortcuts récents, sections scrollables (Récemment joués, Faits pour vous, Mis en avant, Nouvelles sorties)
- **Recherche** : titres, albums, artistes en temps réel
- **Vue playlist / album / artiste** : cover, tracklist, lecture directe
- **Lecteur fixe** en bas : pochette, titre, contrôles (play/pause/skip/shuffle/repeat), barre de progression, volume, appareil actif
- Mise à jour en temps réel via WebSocket (`useSpotifyPlayerWs`)
- Deep-link URL (`?pl=`, `?al=`, `?ar=`)
- Responsive : desktop + mobile

/eq

Égaliseur paramétrique temps réel :

- Visualisation des bandes EQ actives
- Activation / désactivation par bande (checkbox ON)
- Suppression individuelle de bande (bouton poubelle)
- Mise à jour en temps réel via WebSocket

/eqconfig

Gestion des presets EQ :

- Sample rate, fichier de config, layout d'entrée
- Mapping canal par canal Input → Output
- Tableau scrollable, découpé en blocs si > 12 canaux

/lecteurs

Liste de tous les lecteurs audio. Chaque lecteur a un type et une configuration spécifique. Démarrage / arrêt depuis la liste.

/devices

Liste des appareils de sortie. Panneau latéral pour le contrôle de volume, EQ et état en temps réel.

/snap

Interface de contrôle Snapcast (groupes, clients, volume, latence).

/bt

Contrôle des appareils Bluetooth (connexion, déconnexion, état).

Types de lecteurs

Type	Description	Config notable
spotify	Client Spotify (librespot)	bitrate, device-type, typeStream
deezer	Client Deezer	bitrate, device-type, typeStream
radio	Flux radio HTTP	URL du flux, typeStream
local	Fichier audio local	source_path, loop, typeStream
localInput	Capture audio ALSA	pcm_device, layout sortie, master_gain_db

Transports disponibles

Transport	Description
localStream	Sortie sur device audio local (ALSA)
StreamOutFifo	Sortie vers pipe FIFO (ex. Snapcast)
vban	Envoi réseau via protocole VBAN

API backend consommée

Authentification

Méthode	Endpoint	Rôle
POST	/login	Authentification → <code>access_token</code>
POST	/logout	Déconnexion

Lecteurs

Méthode	Endpoint	Rôle
GET	/lecteur	Liste des lecteurs
GET	/lecteur/:id	Détail d'un lecteur
POST	/lecteur	Créer un lecteur
PUT	/lecteur/:id	Modifier un lecteur
PUT	/lecteur/:id/start	Démarrer
PUT	/lecteur/:id/stop	Arrêter
DELETE	/lecteur/:id	Supprimer

EQ

Méthode	Endpoint	Rôle
GET	/eq	Liste des presets
GET	/eq/:id	Détail d'un preset
POST	/eq	Créer un preset
PUT	/eq/:id	Modifier un preset
DELETE	/eq/:id	Supprimer un preset

Spotify

Méthode	Endpoint	Rôle
GET	/spotify/me	Profil utilisateur
GET	/spotify/me/tracks	Titres likés
GET	/spotify/me/player/recently-played	Historique de lecture
GET	/spotify/playlists/me	Playlists de l'utilisateur
GET	/spotify/playlists/:id	Détail d'une playlist
GET	/spotify/albums/:id	Détail d'un album
GET	/spotify/artists/:id	Détail d'un artiste

Méthode	Endpoint	Rôle
GET	/spotify/artists/:id/top-tracks	Top titres artiste
GET	/spotify/artists/:id/albums	Albums d'un artiste
GET	/spotify/search/tracks	Recherche titres
GET	/spotify/search/albums	Recherche albums
GET	/spotify/search/artists	Recherche artistes
PUT	/spotify/devices/play	Lecture (URI / context)
PUT	/spotify/devices/pause	Pause
POST	/spotify/devices/next	Titre suivant
POST	/spotify/devices/previous	Titre précédent
PUT	/spotify/devices/seek	Seek (position ms)
PUT	/spotify/devices/volume	Volume appareil
PUT	/spotify/devices/shuffle	Shuffle on/off
PUT	/spotify/devices/repeat	Repeat (off/context/track)

WebSockets

Canal	Rôle
ws://.../lecteur	États temps réel des lecteurs (start/stop/error)
ws://.../controlOfDevice	Contrôle et état des appareils (volume, EQ)
ws://.../spotify/player	État lecteur Spotify (track, position, volume, device)
ws://.../snap	État Snapcast (groupes, clients)

Protocole JSON : { method: "...", params: [...] }

Authentification

- Token Bearer stocké dans un cookie **TOKEN** (sameSite: lax)
- Middleware global : redirige vers /login si pas de token
- Réponses 401/402/403 → logout automatique + redirection /login

Thème

- Couleur primaire : **green**
- Couleur neutre : **zinc**

- Mode clair / sombre : toggle sur la page de login, préférence système ensuite
-

Commandes utiles

```
pnpm lint          # Linting ESLint
pnpm typecheck     # Vérification TypeScript (vue-tsc)
```